LAB SESSION 02

Singly Linked List Operations

THEORY

A singly linked list is a dynamic linear data structure made up of nodes, where each node contains:

- Data (the actual value)
- A pointer (or reference) to the next node in the list

Unlike arrays, linked lists do not use contiguous memory. Each node is allocated memory separately and is linked using pointers. This provides flexibility in memory usage and makes insertion and deletion more efficient compared to arrays, especially in scenarios where memory needs to be allocated/deallocated frequently. Fig. 2.1 shows a basic structure of a singly linked list.

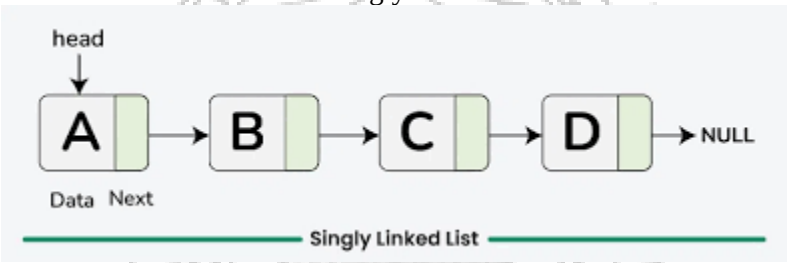

Fig. 2.1. Singly Linked List

Structure of a Node:

```
struct Node {
   int data;
   Node* next;
};
```

Key Operations:

- **Insertion** – At the beginning, middle, or end
- **Deletion** – Remove a node from the list
- **Traversal** – Visit each node and perform an action (e.g., print)
- **Search** – Find a node containing a specific value

Advantages:

- **Dynamic size** – Can grow/shrink as needed
- **Efficient insertion/deletion** – Especially at the beginning

Disadvantages:

- **No random access** – O(n) time to access elements
- **More memory usage** – Extra pointer in each node

- **Sequential access only**

Applications:

- Implementing stacks and queues
- Dynamic memory management
- Adjacency lists in graph algorithms
- Polynomial arithmetic

PROCEDURE

1. Define a Node structure containing data and next.
2. Create multiple nodes and link them using pointers.
3. Traverse the linked list to display its elements.
4. Implement insertion (at beginning or end) and deletion operations.
5. Compile and run the code to validate output.

CODE

```
#include <iostream>
using namespace std;

struct Node {
   int data;
   Node* next;
};

void printList(Node* head) {
   Node* temp = head;
   while (temp != NULL) {
      cout << temp->data << " -> ";
      temp = temp->next;
   }
   cout << "NULL" << endl;
}

int main() {
   // Create nodes
   Node* head = new Node();
   Node* second = new Node();
   Node* third = new Node();

   // Assign data and links
   head->data = 10;
   head->next = second;

   second->data = 20;
   second->next = third;

   third->data = 30;
```

```
    third->next = NULL;

    // Print the linked list
    printList(head);

    return 0;
}
```

EXPECTED OUTCOME

10 -> 20 -> 30 -> NULL

EXERCISE

1. Write a function to insert a new node at the beginning.
2. Write a function to delete the last node in the list.
3. Count the total number of nodes in the linked list.
4. Write a function to search for a value in the linked list.
5. Write a function to reverse the linked list **iteratively**.
6. Write a function to sort the linked list in **ascending order**.